Plan de contingencia por fallas de la base de datos — Food Store

Proyecto: Food Store (Grupo 3) · Bases de Datos II · PostgreSQL 16+ · **Versión:** 1.0, 22/10/2026

1. Objetivo y alcance

Este plan define qué se hace **antes** de una falla, **durante el uso normal** y **cuando la falla ocurre** para recuperar la base `food_store` con una pérdida de datos y un tiempo fuera de servicio acotados. Cubre la base, la configuración del servidor y las copias de respaldo. Completa `SPEC_U6.md` y usa los scripts de esta carpeta: `backup_foodstore.ps1`, `restore_test.ps1`, `validacion_restore.sql`, `health_check.sql` y `maintenance.sql`.

- **RPO** (punto de recuperación): cuántos datos se pueden perder. Equivale a la antigüedad del último respaldo utilizable.
- **RTO** (tiempo de recuperación): cuánto tiempo puede estar el servicio sin funcionar, desde que se detecta la falla hasta que se vuelve a operar.

2. Antes: preparación

2.1 Qué se respalda

Elemento	Cómo se respalda	Por qué
Datos y objetos de food_store	backup_foodstore.ps1 (pg_dump -Fc) más el .conteo.txt	Tablas, vistas, funciones, triggers, índices y permisos (GRANT)
Roles del servidor	pg_dumpall --globals-only -f globals.sql	pg_dump no incluye roles. Sin este archivo se pierden app_web, admin_datos, rol_reportes y los demás roles de la Práctica 7
Configuración	Copia de postgresql.conf, postgresql.auto.conf y pg_hba.conf	La auditoría de la Práctica 7 se activó con ALTER SYSTEM, que se guarda en postgresql.auto.conf
Esquema y scripts	Repositorio en GitHub	schema.sql, objects.sql y los scripts ya tienen una copia remota
Versión del motor	Anotar la versión exacta (hoy 18.6) y guardar el instalador	pg_restore tiene que ser de la misma versión que pg_dump o más nueva

2.2 Dónde se guarda: regla 3-2-1-1

Cada respaldo tiene que existir en **3 lugares**, en **2 soportes físicos distintos**, con **1 copia fuera del lugar** y **1 copia desconectada o inmutable**:

Copia	Ubicación	Protege contra	Estado
1	Carpeta backups\ del servidor	Errores lógicos; es la restauración más rápida	Implementado
2	Segundo disco físico u otro equipo del equipo (otra partición del mismo disco no sirve)	Falla del disco o del servidor	A implementar
3	Nube con historial de versiones (Google Drive, OneDrive o S3 con versionado), cifrada	Robo, incendio o pérdida total del lugar	A implementar
4	Disco externo que se conecta una vez por mes y se desconecta después de copiar	Ransomware y borrados que se sincronizan	A implementar

Una carpeta sincronizada no es un respaldo. Si el archivo local se borra o se cifra, el cambio también se sincroniza. Por eso hacen falta el historial de versiones de la nube y la copia desconectada.

2.3 Seguridad de las copias

- Los `.dump` contienen datos personales (mail, celular y contraseña de `usuario`). Antes de salir del servidor se cifran con la **clave pública GPG** del equipo: `gpg --encrypt --recipient foodstore-respaldos foodstore_X.dump`. El script no necesita ninguna contraseña, y solo quien tiene la clave privada puede descifrar. `globals.sql` contiene los hashes de las contraseñas de los roles y se trata igual.
- Junto a cada copia se guarda su hash: `Get-FileHash foodstore_X.dump -Algorithm SHA256`. Así se puede comprobar que la copia no se corrompió al moverla.
- Las contraseñas van en un gestor de contraseñas compartido, nunca en scripts ni en el repositorio. Las tareas programadas leen la contraseña de `%APPDATA%\postgresql\pgpass.conf`, en la cuenta del usuario que ejecuta la tarea.
- Los respaldos con datos reales nunca se suben a GitHub. En el laboratorio sí se versiona el `.dump`, porque contiene solo datos de ejemplo.

2.4 Personas y documentación

- **Al menos dos integrantes** saben restaurar y tienen acceso a la nube, al gestor de contraseñas y a la clave privada GPG. Si uno no está, el otro puede hacerlo.
- Roles durante un incidente: **responsable** (coordina, elige el punto de recuperación y comunica), **operador** (ejecuta la restauración) y **verificador** (corre las validaciones y controla los pasos). Una persona puede cubrir dos roles, pero quien restaura no valida su propio trabajo.
- Este plan se guarda también fuera del servidor (en la nube y en papel): si el servidor se cae, el plan tiene que poder leerse igual.

3. Durante el uso: respaldos y controles

3.1 Frecuencia de respaldo según cuántos datos se aceptan perder

Nivel	Cómo se respalda	RPO	Cuándo usarlo
1. Laboratorio (actual)	pg_dump diario a las 02:00	≤ 24 h	Base de prueba con datos de ejemplo
2. Inicio de operación	pg_dump cada 1 h en horario de atención, más el diario	≤ 1 h	Pocos pedidos por día. Una base chica se respalda en segundos (hoy pesa 22 KB)
3. Producción	pg_basebackup semanal más archivado continuo de WAL	≤ 5 min	Pedidos durante todo el día. Permite volver a un minuto exacto (PITR)

Configuración del nivel 3. `archive_mode` requiere reiniciar el servicio, y la carpeta de WAL tiene que estar en **otro disco** que los datos:

```
wal_level = replica
archive_mode = on
archive_command = 'copy "%p" "F:\\wal_archive\\%f"'
archive_timeout = 300          # como maximo 5 min sin archivar
```

3.2 Calendario de tareas

Frecuencia	Tarea	Evidencia de que salió bien	Estado
Diaria, 02:00	backup_foodstore.ps1 desde el Programador de tareas	.dump y .conteo.txt nuevos, log sin ERROR	Script listo, falta programarlo
Diaria, después del respaldo	pg_restore --list (el archivo se puede leer), hash SHA-256 y copia al segundo disco (robocopy) y a la nube (.gpg)	El archivo está en los 3 lugares con el mismo hash	A implementar
Diaria	health_check.sql; espacio libre en disco; revisar alertas	Ninguna tarea terminó con código 1	Script listo
Antes de cada cambio riesgoso	Respaldo manual y punto de restauración con nombre (ver abajo)	Respaldo con fecha anterior al cambio	Procedimiento
Semanal	restore_test.ps1 y maintenance.sql	EXITO en restore_test_log.txt	Script listo
Mensual	Restaurar una copia bajada de la nube; copia al disco desconectado; limpieza por retención	La copia remota también se puede restaurar	A implementar
Semestral	Simulacro completo de un escenario C o D, con cronómetro	Informe post-incidente, como informe_u6	Procedimiento

`robocopy` termina con código 0 a 7 cuando la copia sale bien. Un código 8 o mayor indica error. Los scripts tienen que tratarlo así, porque si no un 1 ("se copiaron archivos") se toma como falla.

Puntos de restauración. Se hacen antes de migraciones de esquema, `UPDATE` o `DELETE` masivos, cargas de datos y actualizaciones de PostgreSQL. Siempre se corre un respaldo manual (`.\backup_foodstore.ps1`). En el nivel 3 también se marca el punto en el WAL con `SELECT pg_create_restore_point('antes_migracion_2026_10_30');`, y después se puede volver exactamente a ese momento.

3.3 Retención

Se conservan los últimos **7 respaldos diarios**, **4 semanales** (los domingos) y **6 mensuales** (el primero de cada mes). En el nivel 3 se conservan los WAL desde el respaldo base más antiguo que se quiera poder usar. Nunca se borra un respaldo sin antes confirmar que hay uno más nuevo que se restauró correctamente.

4. Ante una falla: procedimiento de recuperación

Este procedimiento parte de que los pasos de las secciones 2 y 3 se cumplieron, es decir, que hay copias verificadas en varios lugares.

4.1 Pasos comunes, siempre en este orden

1. **Detectar y declarar.** Anotar la hora (ahí arranca el RTO), quién detectó la falla y qué síntoma tiene. Avisar al responsable.
2. **Contener, sin empeorar la situación.** Detener la aplicación. Si no se puede, bloquear las conexiones desde la base `postgres` con `ALTER DATABASE food_store ALLOW_CONNECTIONS false;` y cerrar las sesiones abiertas con `pg_terminate_backend`. **No borrar** archivos del directorio de datos, en especial los de `pg_wal`, ni siquiera para liberar espacio. No usar `pg_resetwal`. Si el servidor está detenido, copiar el directorio de datos completo y los logs antes de tocar nada.
3. **Diagnosticar.** Ubicar la falla en uno de los escenarios de la sección 4.2 y elegir el punto de recuperación. En un error lógico, el punto es el último respaldo **anterior al error**, no el más reciente.
4. **Verificar la copia antes de usarla.** El hash tiene que coincidir con el `.sha256` y `pg_restore --list` tiene que leer el archivo sin errores.
5. **Restaurar en una base nueva**, nunca encima de la dañada (ver sección 4.3).
6. **Validar.** Correr `validacion_restore.sql` y comparar con el `.conteo.txt` del respaldo. Correr también `health_check.sql`, `ANALYZE` (el respaldo no incluye las estadísticas del planificador: `pg_dump` 18 solo las guarda con `--statistics`) y `SELECT * FROM v_pedidos_resumen;`.
7. **Volver a operar.** Intercambiar los nombres de las bases, habilitar las conexiones y levantar la aplicación.
8. **Recuperar lo perdido y comunicar.** Los pedidos registrados entre el respaldo y la falla (la ventana del RPO) se reconstruyen con los comprobantes, la plataforma de pago o los mails de confirmación. Avisar a los usuarios afectados.
9. **Cerrar.** Anotar la hora de fin (RTO real), hacer un respaldo nuevo apenas se vuelve a operar y escribir el informe post-incidente con una mejora concreta. Si hubo acceso indebido, cambiar todas las contraseñas.

4.2 Escenarios

Escenario	Ejemplos	Desde dónde se recupera	RTO objetivo	Qué tiene de particular
A. Error lógico	DELETE o UPDATE sin WHERE, un bug, una carga errónea	Copia local, o PITR en el nivel 3	≤ 1 h	El servidor funciona. Se restaura en una base aparte y se copian solo las filas afectadas. Las bajas que hace la aplicación son lógicas (eliminado = TRUE): si se identifican las filas, se revierten con un UPDATE, sin restaurar nada
B. Base dañada o servicio caído con el equipo sano	El servicio no arranca, hay corrupción o el disco está lleno	Copia local	≤ 1 h	Liberar espacio fuera de la carpeta de datos. Si el servicio no arranca, crear un clúster nuevo y restaurar
C. Falla física del servidor	Disco roto, falla de la fuente, equipo inutilizable	Segundo disco u otro equipo	≤ 4 h	Instalar en otro equipo la misma versión de PostgreSQL y restaurar primero globals.sql y la configuración
D. Pérdida total o ataque	Robo, incendio, ransomware, borrado de todas las carpetas	Nube con historial de versiones o disco desconectado	≤ 1 día	Usar un equipo limpio. Restaurar una versión anterior al ataque, descifrar con la clave GPG y cambiar todas las contraseñas

El RTO de 10 minutos de `SPEC_U6.md` es el tiempo técnico de restaurar y validar (en el simulacro se midió 0,99 s). Los objetivos de esta tabla suman además la detección, el diagnóstico y la preparación del equipo.

4.3 Comandos de recuperación (escenarios B, C y D)

```
# 1. Verificar la copia (en C y D, primero descifrar: gpg --output X.dump --decrypt X.dump.gpg)
$dump = ".\foodstore_20261022_021004.dump"     # ultimo respaldo anterior a la falla
Get-FileHash $dump -Algorithm SHA256
pg_restore --list $dump > $null
# 2. Solo en un servidor nuevo: roles del servidor
psql -h localhost -p 5432 -U postgres -d postgres -f globals.sql
# 3. Base nueva. A diferencia del ensayo semanal, se restaura CON duenos y permisos
createdb -h localhost -p 5432 -U postgres food_store_recuperada
pg_restore -h localhost -p 5432 -U postgres -d food_store_recuperada --exit-on-error $dump
# 4. Validar (paso 6) y cambiar de base, conectado a la base postgres y sin sesiones abiertas
ALTER DATABASE food_store RENAME TO food_store_danada;      -- solo si todavia existe
ALTER DATABASE food_store_recuperada RENAME TO food_store;
```

La base `food_store_danada` se conserva hasta terminar el informe post-incidente, porque puede tener datos posteriores al respaldo que se pueden rescatar. En el ensayo semanal se usa `--no-owner --no-privileges` porque solo se valida el contenido. En una recuperación real los roles ya fueron restaurados con `globals.sql`, así que se conservan los dueños y los GRANT de la Práctica 7.

4.4 Recuperación a un momento exacto (PITR, solo nivel 3)

1. Detener el servicio y mover el directorio de datos dañado a otra carpeta, sin borrarlo.
2. Extraer en un directorio de datos vacío el último `pg_basebackup` **anterior** al punto elegido.
3. En `postgresql.conf`, definir `restore_command = 'copy "F:\\wal_archive\\%f" "%p"'`, el punto de llegada (`recovery_target_time = '2026-10-30 14:29:00-03'` o `recovery_target_name = 'antes_migracion_2026_10_30'`) y `recovery_target_action = 'pause'`.
4. Crear un archivo vacío `recovery.signal` en el directorio de datos e iniciar el servicio.
5. Al llegar al punto, el servidor queda en pausa y en solo lectura. Validar los datos y, si el punto es el correcto, ejecutar `SELECT pg_wal_replay_resume();` para terminar la recuperación.

PITR lleva **todo el servidor** (todas sus bases) a ese minuto. En un error lógico conviene hacerlo en otro equipo, o en otro directorio y otro puerto, y copiar desde ahí solo los datos afectados.

5. Mantenimiento del plan

El plan funciona solo si se prueba. `restore_test.ps1` comprueba cada semana que el último respaldo se puede restaurar. El simulacro semestral prueba el procedimiento completo con personas y tiempos reales. El plan se revisa después de cada incidente o simulacro, cuando cambia la versión de PostgreSQL o el esquema, y cuando cambian los integrantes que tienen acceso a las copias.